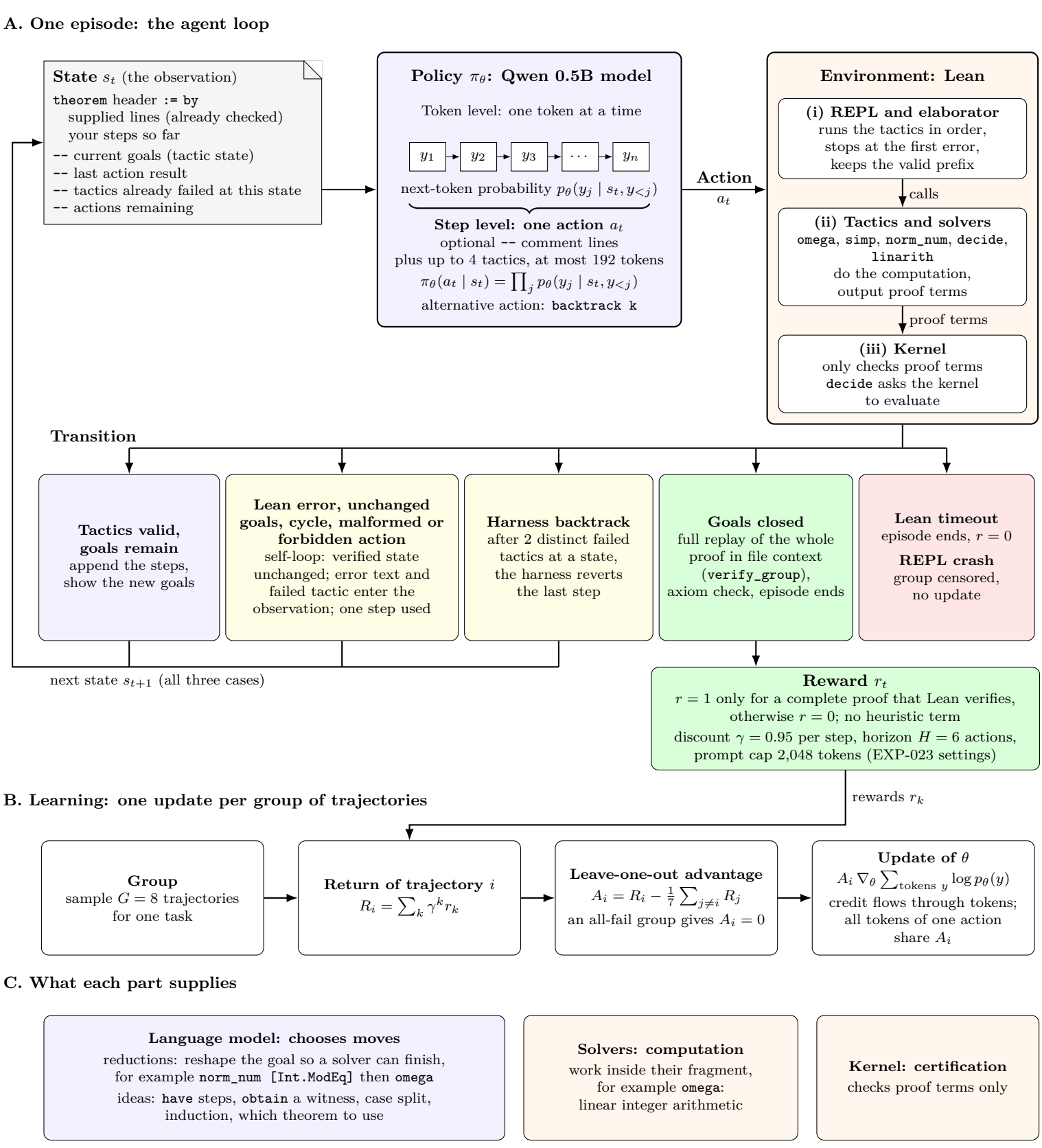

A. One episode: the agent loop
State s_t (the observation)
theorem header := by
supplied lines (already checked)
your steps so far
-- current goals (tactic state)
-- last action result
-- tactics already failed at this state
-- actions remaining
Policy π_θ: Qwen 0.5B model
Token level: one token at a time
y_1 y_2 y_3 $\cdots$ y_n
next-token probability $p_\theta(y_j \mid s_t, y_{<j})$
Step level: one action a_t
optional -- comment lines
plus up to 4 tactics, at most 192 tokens
$\pi_\theta(a_t \mid s_t) = \prod_j p_\theta(y_j \mid s_t, y_{<j})$
alternative action: backtrack k
Action
a_t
Environment: Lean
(i) REPL and elaborator
runs the tactics in order,
stops at the first error,
keeps the valid prefix
calls
(ii) Tactics and solvers
omega, simp, norm_num, decide, linarith
do the computation,
output proof terms
proof terms
(iii) Kernel
only checks proof terms
decide asks the kernel
to evaluate
Transition
Tactics valid, goals remain
append the steps,
show the new goals
Lean error, unchanged goals, cycle, malformed or forbidden action
self-loop: verified state
unchanged; error text and
failed tactic enter the
observation; one step used
Harness backtrack
after 2 distinct failed
tactics at a state,
the harness reverts
the last step
Goals closed
full replay of the whole
proof in file context
(verify_group),
axiom check, episode ends
Lean timeout
episode ends, $r = 0$
REPL crash
group censored,
no update
next state s_{t+1} (all three cases)
Reward r_t
$r = 1$ only for a complete proof that Lean verifies,
otherwise $r = 0$; no heuristic term
discount $\gamma = 0.95$ per step, horizon $H = 6$ actions,
prompt cap 2,048 tokens (EXP-023 settings)
B. Learning: one update per group of trajectories
rewards r_k
Group
sample $G = 8$ trajectories
for one task
Return of trajectory i
$R_i = \sum_k \gamma^k r_k$
Leave-one-out advantage
$A_i = R_i - \frac{1}{7} \sum_{j \neq i} R_j$
an all-fail group gives $A_i = 0$
Update of θ
$A_i \nabla_\theta \sum_{\text{tokens } y} \log p_\theta(y)$
credit flows through tokens;
all tokens of one action
share A_i
C. What each part supplies
Language model: chooses moves
reductions: reshape the goal so a solver can finish,
for example norm_num [Int.ModEq] then omega
ideas: have steps, obtain a witness, case split,
induction, which theorem to use
Solvers: computation
work inside their fragment,
for example omega:
linear integer arithmetic
Kernel: certification
checks proof terms only